1. Tecnologias e Arquitetura

- **Linguagem:** PHP com Programação Orientada a Objetos (POO).
- **Banco de Dados:** MySQL ou MariaDB.
- **Conexão:** Exclusivamente via **PDO**.
- **Arquitetura:** MVC (Model, View, Controller) rigorosamente separada.
- **Autenticação:** Controle de acesso via Sessões do PHP.

2. Estrutura do Banco de Dados (turcar.sql)

O sistema deve ter pelo menos as seguintes tabelas (e no mínimo **8 veículos pré-cadastrados**).

Tabela usuarios

- id (PK)
- nome
- cpf
- data_nascimento
- e_mail
- telefone
- endereco, numero, cidade, estado
- senha (Armazenada como hash)

Tabela carros

- id (PK)
- marca, modelo, versao, ano
- preco (Decimal)
- quilometragem, cor, combustivel, cambio, numero_portas
- descricao
- imagem (Caminho/URL da imagem)
- quantidade_disponivel / status

Tabela carrinho

- id (PK)
- usuario_id (FK -> usuarios.id)
- carro_id (FK -> carros.id)
- quantidade

3. Regras de Negócio e Validações no Servidor (PHP)

Validação de Cadastro e Idade:

- **Maioridade:** O PHP deve validar a data_nascimento. Se for menor de 18 anos, o cadastro deve ser bloqueado no back-end.
- **Dados:** Validar preenchimento de campos obrigatórios, formato de e-mail, quantidade mínima de caracteres na senha (ex: 8) e confirmação de senha.

Autenticação e Sessões:

- **Login:** Buscar usuário pelo e-mail e validar a senha (ver seção de Segurança).

- **Sessão:** Ao logar, criar uma $_SESSION identificando o usuário.
- **Controle de Acesso (Rotas Protegidas):** Redirecionar para o login tentativas de acesso a áreas restritas (ex: Carrinho, Finalizar Compra) por usuários não autenticados.
- **Logout:** Destruir a sessão ao sair.

Carrinho e Pagamento:

- **Cálculos:** O back-end deve calcular o subtotal por item e o valor total do carrinho.
- **Ações do Carrinho:** Inserir, remover, alterar quantidade e limpar carrinho.
- **Simulação de Parcelamento:** Lógica no PHP para calcular parcelas (ex: 12x, 24x, 36x, 48x, 60x), aplicando taxas de juros caso a equipe defina.

Filtros/Busca (Opcional recomendado)

- Lógica no back-end para filtrar carros no banco com base em: marca, modelo, ano, faixa de preço, combustível e câmbio.

4. POO e Estrutura MVC

Classes de Domínio (POO):

As classes devem usar encapsulamento, atributos, métodos e construtores apropriados:

- Usuario
- Carro
- Carrinho
- Pagamento
- Conexao (Centraliza a instância do PDO)

Controladores e Modelos Esperados:

- **Models:** Usuario.php, Carro.php, Carrinho.php (Fazem o CRUD e lógica de banco).
- **Controllers:** UsuarioController.php, CarroController.php, CarrinhoController.php, AuthController.php (Recebem requisições, chamam os models e preparam dados para as views).

5. Segurança

- **Prevenção de SQL Injection:** Uso obrigatório de *Prepared Statements* do PDO em todas as consultas.
- **Criptografia de Senhas:**
  - No cadastro: Usar password_hash().
  - No login: Usar password_verify().
  - **Proibido:** Armazenar senhas em texto puro.
- **Validação Dupla:** Mesmo que o front-end valide, o PHP **deve** validar os dados antes de inserir no banco.

6. Estrutura de Pastas (Foco Back-end)

Sugestão de árvore para a organização exclusiva dos arquivos de configuração e back-end:
Plaintext

```
/concessionaria
├── /app
│   ├── /controllers
│   │   ├── AuthController.php
│   │   ├── CarroController.php
│   │   ├── CarrinhoController.php
│   │   └── UsuarioController.php
│   └── /models
│       ├── Carro.php
│       ├── Carrinho.php
│       └── Usuario.php
├── /config
│   └── conexao.php      # Classe PDO
└── /database
    └── concessionaria.sql # Estrutura de tabelas e os 8 carros iniciais
```